

小团队 AI 组织变革的实验与思考

年初随着 Open Claw 的走红，AI 从陪伴聊天到帮忙干活，带来了许多“新范式”的讨论。无论是 Agent Harness 的工程化探索，还是各种 AI 组织变革的实验，都在这个方向上持续摸索。今年 Codex 和 PeterPang 团队“AIFirst”的实践，似乎让新范式从抽象变得有迹可循：不要试图用 AI 优化每一个流程，而是先假设这件事应该由 AI 完成，再决定哪些环节留给人做配合。人不再是流程的中心，Agent 才是。虽然激进，但也好用。

X 上各种创业团队和 OPC 的实践，也让这个方向逐渐清晰起来：扁平化、高自治的小团队，将成为 AI 时代的基本组织单元，组织单元不再是岗位，而是“Agent+人”的混合体。结合晚点对俞浩的专访，让我想起韩都衣舍十多年前的“产品小组制”，三个人一组（设计师、买手、视觉）对一个品类负责到底，每个小组拥有从选款、定价、生产、销售的完整决策权，用数百个自治小团队的“小单快返”应对当时电商对服装行业改造变化。追觅近几年“孵化器+BU”组织模式是同样的逻辑，扫地机、洗地机、吹风机切成独立小 BU，自负盈亏、独立招聘、内部赛马。把这些样本拼到一起，会发现它们指向同一个事实：组织形态本身，正在成为 AI 时代生产力的真正瓶颈。福特留下的科层制，靠信息差建立起来的决策机制，正在逐渐丧失效力。如果把 AI 当成外挂工具，装在原本的流程和组织上，生产力会被旧组织框住，还停留在“蒸汽机加固水轮”的阶段。另外，最近越发感受到 AI 组织变革本质上就是 harness 工程在代码层和组织层的层层展开，harness 为 Agent 做系统设计的同时，也需适配“Agent+人”的混合工作模式，两层做的是同一件事，只是用了不同的语言。

今年随着外卖行业环境变化，叠加 AI 浪潮，我们团队做了一些有趣的组织实验。卫星店团队是外卖成熟业务下的子业务单元，偏供给方向探索，属于高不确定性、高模糊度、需要快速试错，要不断从行业拿认知、靠实践验错。AI 恰好贴合我们这种处境，给了“Agent+小团队”试验的土壤，团队逐渐形成凡是模糊的、不会的、没经验的，就学 OPC 模式让 Agent 带着人做，人来抛问题转化 coding 信息。

1、最小运营单元的 Agent 实验

卫星店是大正餐品牌开的外卖店型，主做外卖生意。这类品牌在运营上有一个结构性矛盾，他们擅长线下餐厅管理服务，但卫星店的生意主要靠外卖，而外卖流量运营恰恰是他们最薄弱的环节，要么没人、要么有人但没水平。过去我们站在平台视角是不太相信这件事的，心想这么大的牌子还不会投流？直到我们团队自己下场运营门店才知道是真不会。我反思过，过去我们用的是“按模块分工”的方式：平台运营按区域切片，每个人负责一片区域，通过总部→销售→商家的三层传递，提升商家经营水平。每多一层传导，信息折损一次、目标/激励稀释一次，于是陷入了“做得越精细、覆盖规模越窄”的两难。

去年 12 月，我们某个新项目需要团队亲自操刀线上运营“算是给了我们机会”，面对从没做过的事情，一开始团队边学边干、按惯性复盘迭代。随着项目扩大，从投入 1 个人变成 2 个、3 个，逐渐发现这事儿不对，陷入我们不擅长且怎么做都不会比商家做好的“陷阱”中。那时候我们决定换一种干法：把这件事交给 AI 干，不一定比我们干得更差。我们自己手搓了一个自动化投流脚本，一开始只完成“重复投放计划”这一件事。在这过程中，我们的思维发生了变化，逐渐把投放效果、策略、分析、复盘也都期待于 Agent 完成（人确实都是懒的），

不断抛问题让 Alcoding 输出解法。从一个自动化投放脚本,慢慢搓成了一个智能投放 Agent,能自动扫描门店流量水平、诊断问题、生成策略、执行投放、回收数据、再调整。后来我们发现,Agent 的策略密度和反应速度,已经超过了一个普通同学能投入的精力,也不需要投入同学了,那个同学只需要继续手搓这个 Agent。

这次小实验,除了效率提升之外,更关键的是岗位本身变了。原本“门店运营”的岗位变成了 Agent 的“教练”,从几个人负责一件事,变成一个教练带 Agent 团队干一件事,而且这个教练还是业余的。

2、context 是组织基础

把 Agent 当成团队成员,远比“装个工具”复杂得多。一个真正能参与团队思考的 Agent,需要和团队成员所拥有的信息一样多,否则大模型再聪明,也是在信息黑箱里推理,推出来的东西没人敢用,也无法真正做到围绕它来做组织建设。4 月开始,我们在团队里推行一件事,内部叫“小美世界模型”的 Context 工程,目的让 Agent 能拿到的信息和团队成员拿到的尽可能一样多。我们把输入的信息被分成三类:

第一类,数据。数据是 Agent 最容易获得的内容,权限规则上也最清晰;结构化的底表接入相对直接,权限也能依附同学权限。

第二类,文字。面向 Agent 建立它习惯读的知识库(prompt 基础)。这件事一开始比较麻烦,历史文档若散落在各个空间、没有规则管理,整合起来难度较大。我们尽量挑选已有的、按规则新增的,集中到一个 Agent 可读的知识库里。

第三类,技能,也就是所谓的 skill,员工会什么不会什么。

第三类,交互,即日常讨论和会议,也是最容易被忽视的。这一类信息的密度最高,是团队真正的“思考现场”。如果能被 Agent 利用,发挥的效应最强,比如客户拜访里的吐槽建议、日常讨论中抛出的假设、汇报中的不同观点。这些信息更像信号,埋着重要的思路线索,但过去都消失在记忆里。我们做的方法很笨,但目前看起来有效,所有一线同学参与的会议、讨论、客户拜访,尽可能做录音转文字,形成知识库文档让 Agent 阅读。

当 Agent 和团队成员的信息密度趋近一致,它就能用大模型的思考能力帮团队做更深的判断,是实现 Agent 为中心、人为辅助的基础,围绕 Agent 建设的 AI 组织才有可能。现在回头看,最早投流 Agent 其实就是我们第一次摸索出来的糙版 harness,它有清晰的角色定义、工具(投放管理、数据诊断)、护栏(预算上限、ROI 约束)、错误处理(盯盘报警)以及人机之间的协作协议(人定目标、Agent 定策略&执行)。只是当时我们没意识到自己在做 harness,事后才发现这件事真正难的不是写出脚本而是在逐渐围绕 Agent 作为工作主体下写出的这一圈“挽具”。现在做团队 context 感受更深,context 仅是 harness 工程里“信息输入”那一层,决定了 Agent 思考的上限,但 harness 远不止于此,落地组织层面还会有更难的实践,我们才刚开始啃。

3、“Agent+人”团队项目制

投流 Agent 跑通之后，门店投流这件事就再也没人管过；context 跑起来后，Agent 输出了项目的思路方向，有一件事在团队里慢慢清晰起来，Agent 不只是工具，要把它当成团队一员。按我理解的 Agent harness 的三层结构与之类似：

基础层：模型工具的执行能力，映射到组织上恰是每个项目“Agent+人”的基础能力组合；

中间层：context 工程的 skill、MCP 等，映射到组织上就是让 Agent 与人共享物理世界信息匹配 Agent skill 等；

最上层：编排、协议机制，映射到组织上是对物理世界的团队架构与“Agent+人”的规则编排，搭一套适配 Agent 的组织体系。

这三层中最上层的编排最难，也是现在各种创业团队无法做到的。所以，当时和 HRBP 讨论要用项目制做团队分工时，这句话当时感觉听起来很轻，后来在很多决策点上都开始变重。这里也要感谢 HRBP 团队的支持理解，帮助我们吧 Agent 扩展到整个团队的工作方式：从原有的“模块化分工”切成三五人的小团队，成员没怎么变，但团队内的工作理念和要求发生了变化，不再按人头看而是按“Agent+人”混合体看。和原来小团队最大的不同，是机制让每个人都以 Agent 为中心建设。在这个过程中，我们不断筛选出更有 AI 天赋的同学，也带着轻度使用 AI 的同学被迫跟上（因为要干的事情更多、更不确定）。落到具体动作上，有几个变化：

岗位边界被打散。过去分销运产运，有些擅长数据、有些擅长机制，现在全部拉齐。组织按方向分，每个同学按项目分，打破岗位边界，从“会做某件事”变成“会不会用 Agent 做某件事”。

会议大幅减少。现在团队多人的会议屈指可数，特别是团队开始 Context 后，基本无需“大会”，都被压成一个个同学为自己项目开的各类小会或人机交互。Context 把全组的信息差打破，团队 Agent 每天给所有人共享信息，AI 实时维护进展。

决策链路缩短。从原来一个项目带着一堆人干，到每个人对一个具体项目负责到底，倒逼从调研、试点、迭代到关停或扩大，自己全权决策。组织只定目标、给资源、要结果、加背锅。

这几个月跑下来，有几条经验是我事先没想到的：

项目负责人岗位本身就是一个“放大镜”。过去还模块分工，一个同学往往只能展示局部能力且边界感很强，是否有全链路判断力很少有机会被看到。项目制把原本不属于一个人干的固定模块的活儿，把方向、编排、判断、结果全部压在一个人身上，能力版图被完整暴露，要不失败要么想办法用 Agent 搞定，一旦用 Agent 搞定后就进入了正循环——即更多项目/任务→更多 context (input) →更明确的规范 (PDCA) →更好的 agent 输出 (output) →更多项目/任务。我们这几个月里看到的跑得最好的同学，放在原岗位上几年都不一定会被看见，这是我没料到的副产品。我总说这是天赋。

AI 用得越多，对“人是干什么的”反而要求越高。当大部分执行由 Agent 完成，人贡献的边际价值集中在少数几个判断节点上。任何一个判断的偏差都会被 Agent 放大到整条链路。组织

变薄、人变少，不意味着轻松，人背着 Agent，能力边界被扩展，总会陷入它无所不能的幻觉，它意味着每个人的“能力密度”变得更高。我有时会想“人真的变了吗”，这可能是 AI 时代最反直觉的事不是 AI 让人轻松，而是 AI 让人变贵。

到底是组织变革推动了人的变化，还是人的工作流变化带动了组织变革？这个问题我目前没有答案。从我自己看到的几个变化点回放，更像是“工作流先变”，团队里几个最敏锐的同学，先开始用 AI 重写自己的工作方式，组织被动地然后是主动地跟上去，但很难说这是不是普适规律，还是我们这种探索型业务里才会先这样，或许组织先变革的整体效率更高。

4、一些没搞明白的问题

写到这里，我得诚实地说一件事：这套做法目前最大的卡点，在于我们对 Agent 还没建立起成熟的 harness 工程，也就是上面说的部分中间层问题和全部的第三层。harness 对于组织变革不只是一个比喻，更像是一个指导思想。组织以 Agent 为中心建设，harness 也是一整套围绕 Agent 的工程，两者在纯编程类工作中很容易被一体化看待，但其他涉及到更多物理世界输入与输出类的工作，“Agent+人”中人那个部分该如何去做？前者我们刚跑通，后者还在踩坑。很多问题目前在我们这里都没有现成答案，但行业里其实已经有一些值得借鉴的样本和指导思想，我把它们整理成三条，作为我们接下来的参考坐标：

需要新的 JD“岗位说明书”。PeterPang 团队实践里反复强调，每个 Agent 必须有明确的角色、目标、skill、权限边界，没有边界的 Agent 是不可信的。这件事跟给员工写 JD 是同构的，只不过现在需要写“Agent+人”混合体的 JD，而且要求会更严，人有社会化的常识做兜底，Agent 没有，所有的“理所当然”必须写出来。

Agent+人协作要靠“协议”不能靠“默契”。单 Agent 跑通是一回事，十个 Agent 一起跑是一回事、十个 Agent 和人一起跑是另外一回事。Harness 工程里最难的部分就是编排 orchestration，谁先谁后、谁压谁、冲突时谁仲裁等。这跟物理世界多个团队协作简直一样。换句话说，以 Agent 为中心建设团队的治理难题，本质上是把组织设计的难题拓展到了代码世界和物理世界，但到代码世界这一步还没走通。

可观测性是治理的前提。看不见的 Agent 不可治理，就像藏在团队水下的“默契”“关系”也是不可被规则化的一样，所以“Agent+人”必须有摆到台面的规则制度、奖惩机制等，Agent 数量从 1 个到 100 个的过程中，真正决定组织上限的不是 Agent 数量，而是足够的可预测性。这件事也映射到现有的“科层制”组织里，很多信息差带来的管理和协作，在 AI 时代会变成致命短板。

基于我们现有的不成熟实践，组织进入 AI 时代，第一道坎是人把第一个 Agent 用起来，这道坎我们刚迈过去；第二道坎是 Agent 和人共享思考起来，这道坎我们正在路上；第三道坎也是更深的一道坎，是把第十个、第二十个 Agent 协调起来。后面这道坎，我估计接下来要花掉我们大部分精力去试错。

写下这些的时候，我没有想做总结，也没有想给结论，只是觉得改变开始后，很多事情都是被推着往前走，过去方法追求的是消除不确定性，但一个新世界观下、混沌是无法消除的，下面会发生什么我也不知道，只有全力与最前沿保持最小的距离。